

Present Claude Code Documentation at Scale

A Practical Workflow for Scaling New Industries Using GitHub Repositories

CEO Kevin Kawchak, CC BY. [10.5281/zenodo.19357831](https://doi.org/10.5281/zenodo.19357831)

Top Claude Code Phrases for Avoiding Extra Work

1) "You are responsible for comprehensive understanding and application of all aspects of the current directory"

2) "Only include single dashes: no em dashes, or double dashes, or triple dashes"

3) "Place the new release notes in releases.md. Provide an updated changelog"

4) "Make sure the repository is fully up to date with this work regarding badges, content, and context"

5) "Don't ask questions or go into plan mode throughout this prompt"

6) "There must be three text diagrams, each with their own perspective, for each of the commits"

7) "Update the main Readme diagrams, repository structure, etc."

8) "Provide a copy of this prompt in a new prompts.md"

9) "Only create a new prompt for Claude Code Opus 4.6 (1M context) to process in a future step" (meta prompt)

10) "Provide the exact code files, directories, and additional files that Claude Code will need to access" (meta prompt)

Meta-Prompting: Reducing Errors and Scaling Output

What is Meta-Prompting?

A lot of comments in discussions center around Claude Code errors and token usage. A solution is to first submit a prompt to Claude asking for a new prompt containing processing instructions, and exact directory and file locations for processing the new prompt.

In a separate conversation, submit the new prompt to Claude for more efficient token usage and error reduction that lets Claude scale projects easier, despite the larger input.

This process is called meta-prompting.

Three-Step Workflow: Claude Code at Scale

STEP 1

Meta-Prompt First

Submit a prompt to Claude asking it to “generate a new structured prompt - with exact directory and file locations” - based on your codebase.

STEP 2

Run in a New Conversation

Execute the generated prompt with your codebase. If "massive" appears twice in Claude dialogue, split output into multiple commits in one PR via a revised prompt.

STEP 3

Add a Final Verification Commit

In the same conversation, instruct Claude to include a final commit that confirms all prior commits are correct and properly combined for the PR.



Multiple Commits - Implementation Notes

a) Multiple commits are needed due to limited amounts of working memory per update.

b) A workflow was validated with seven commits in a 90+ page PDF written in LaTeX. Claude paused after each commit, as each update was manually reviewed.

c) A Document featuring an autonomous workflow at 185 pages was generated using two large prompts and over twenty commits. A request can be made for prior commit files not be overwritten with each update.

Complex Workflows & Tips at Scale

Complex Workflows

The trick for scaling projects every week is to let Claude Code do the heavy lifting.

Workflows that are too complex should be simplified and well understood in order for thinking, planning, and running experiments to take most of the time.

Claude Code Tips at Scale

- 1 Avoid moving large amounts of files around the repository
- 2 Pre-chunk text/markdown files over 20K tokens
- 3 Live demo Claude with GitHub in meetups to learn faster

24-Hour Autonomous Simulation Paper

Simulation Design

A 24 hour simulation can be performed using 1 commit per simulation hour at minute resolution. Multiple txt and md files can be outputted per commit; and a final commit can be used for fixes and summarization.

This is done fully autonomously using Claude Code Opus 4.6 (several processing hours) if Claude limitations are well understood, and a substantial prompt is used with [GitHub](#). The final results were implemented into a new 60+ page paper published on [Zenodo](#).

24

commits total
(1 per hour)

1 min

time resolution
per commit

Opus 4.6

model
(1M context)